QVLA: NOT ALL CHANNELS ARE EQUAL IN VISIONLANGUAGE-ACTION MODEL'S QUANTIZATION

26/04/03

[기존 문제]

1. 현재 VLA 모델의 크기

The rapid evolution of foundation models (Touvron et al., 2023a;b) has significantly advanced embodied intelligence, empowering Vision-Language-Action (VLA) models like OpenVLA (Kim et al., 2024) to synthesize complex, executable actions from visual inputs and linguistic directives. These models enhance cross-task generalization and semantic reasoning, elevating the efficacy of robotic manipulation. However, their immense computational and memory demands, which often exceed 14 GB in standard half-precision for a 7B model, present a critical barrier to deployment. For instance, running such a model on a widely-used robotic platform like the NVIDIA Jetson AGX Orin can result in inference latencies of several hundred milliseconds per action, far too slow for the fluid, real-time control required in dynamic environments. This performance gap necessitates aggressive compression methodologies, such as pruning and quantization, to achieve practical inference speeds while sustaining precise control (Yang et al., 2025; Song et al., 2025b; Wen et al., 2024;

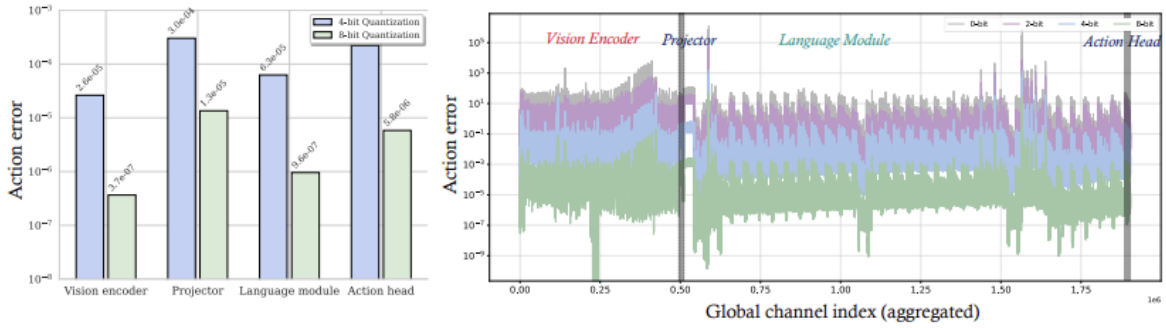
VLA 모델의 크기로 인해, 로봇에 올리지 못하고 클라우드 환경에서 쓸 수밖에 없는 상태. 이로 인해 네트워크 latency 등 추론 속도와 memory 등 현실적 제약 존재.

⇨ **VLA 모델 경량화의 필요성**

2. VLA Quantization

VLA는 action이 누적됨에 따라 최종 task 성공에 대한 오차가 증폭.

또한, VLA는 **LLM decoder(1), vision encoder(2), multi-modal projection layer(3), action decoder(4)** 총 4개의 모듈로 구성됨에 따라, 기존 LLM/MLLM에 썼던 quantization 기법을 적용했을 때 잘 되지 않았음



(a) Quantization Results Across Modules (b) Quantization Results Across Channels

⇨ 모듈 별로, 나아가 channel 별로 action 출력에 대한 민감도가 전부 다름으로 이를 고려하여, **channel-wise quantization**을 적용하자.

[Related Work]

Quantization methods. Model quantization is a cornerstone technique for efficient deep learning deployment, reducing memory footprint and computational latency by representing weights and activations with low-bit integers. The central challenge lies in minimizing the ensuing accuracy degradation. Two primary methodologies dominate the field, *i.e.,* Post-Training Quantization (PTQ) (Frantar et al., 2022; Li et al., 2021; Nagel et al., 2020) and Quantization-Aware Training (QAT) (Jacob et al., 2018; Esser et al., 2020). PTQ offers a low-cost solution by quantizing a pretrained model with a small calibration set and no retraining. In contrast, QAT simulates the effects of quantization during training, allowing the model to adapt its parameters to mitigate precision loss. Recent advances have focused on mitigating the challenges of quantizing large models, particularly the presence of outliers and activation-induced quantization difficulties. Methods like SmoothQuant (Xiao et al., 2022) re-scale weights and activations to create a more favorable quantization landscape, while AWQ (Lin et al., 2023) preserves salient weights and OmniQuant (Shao et al., 2024) smooths weight and activation outliers. To handle pernicious outliers in weights and activations, another research thrust employs learned rotations or permutations to redistribute value distributions, exemplified by methods like QuaRot (Ashkboos et al., 2024) and SpinQuant (Liu et al., 2024c), enabling robust 4-bit quantization. However, a common limitation across these methods is the assumption of uniform precision, where a single bit-width is applied globally or, at best, per layer (*e.g.*, HAWQ (Dong et al., 2019; 2020)). This coarse-grained approach is fundamentally misaligned with the requirements of embodied policies. VLA models exhibit heterogeneous sensitivity across their architecture. For instance, subtle shifts in action-generation layers can lead to catastrophic failures in control, a fragility not typically observed in standard perception or language tasks. Motivated by this critical shortcoming, we propose a fine-grained, action-guided, channel-wise quantization framework.

기존 LLM method들은 모델 전체나 특정 layer 단위로, 동일한 bit 수를 일괄 적용하는 "uniform precision"을 가정

하지만, VLA에서 "Action"에 미치는 민감도를 channel 별로 세밀하게 bit를 할당해야 함

[About VLA]

3.1 PRELIMINARIES

Vision-Language-Action (VLA) models. A VLA model governs the behavior of an embodied agent by defining a policy, denoted as Π, that maps high-dimensional sensory inputs and a language directive to a sequence of actions. The model operates within a sequential decision-making framework. Formally, at each discrete timestep t, the agent perceives its environment through visual observations $\mathcal{V}_t$ (*e.g.*, RGB images) and receives a time-invariant language instruction (prompt) p. The policy's role is to predict an action sequence $\mathcal{A}_t$ (*e.g.*, end-effector poses or joint velocities),

$$\Pi_\theta(\mathcal{A}_t|\mathcal{V}_t, p). \tag{1}$$

The parameter set θ typically comprises: ♠ vision encoders θ_{vis} that processes the high-dimensional visual inputs $\mathcal{V}_t$ into compact feature representations; ♠ a projection layer θ_{proj} that maps these visual features into the multimodal embedding space shared with the language modality; ♠ a Large

간단하게 말해서, 현재 scene에 대한 visual observation과 지시명령 prompt가 주어지면, 그 상황에 맞는 Action을 내뱉는 것이 VLA.

이때 각 timestep별로 sequential하게 action을 수행하고 관측하고 이를 반복.

[Method]

[Sensitivity Analysis]

1. 모듈 별 sensitivity

   (Action decoder/Multi-modal projection)>Language Model>Vision Encoder

   순으로 quantization에 따른 degradation에 민감함

   이에 action decoder는 full precision을 유지하면서, vision encoder와 language model을 중점적으로 quantization 진행.

components serve as the final nexus for translating multimodal understanding into physical action. Any perturbations introduced here propagate directly and without mitigation to the output action distribution, leading to significant and often erroneous deviations in the agent's behavior. These findings underscore that an adaptive quantization strategy is not merely beneficial but essential for preserving the functional integrity of VLA models at low bit-widths.

2. Channel 별 sensitivity

   Per channel 별, mixed-precision strategy 사용

   이때, {16, 8, 4, 2, 0(pruning)} bit 각 조합에 대해 sensitivity check.

   Sensitivity는 다음과 같이 각 time step 별과, 전체 episode 누적에 대한 sensitivity을 구할 수 있음

   Sensitivity는 action space에 대한 quantization error와 같음

To effectively identify these salient channels, we propose a novel sensitivity metric grounded directly in the action space rather than the intermediate feature space. More specifically, for a given layer l and a specific channel c within it, we isolate its impact by quantizing only that channel to a bit-width $b \in \{0, 2, 4, 8, 16\}$, while all other parameters remain in full precision. We then define the single-step action sensitivity as the expected squared L2 norm of the resulting action deviation,

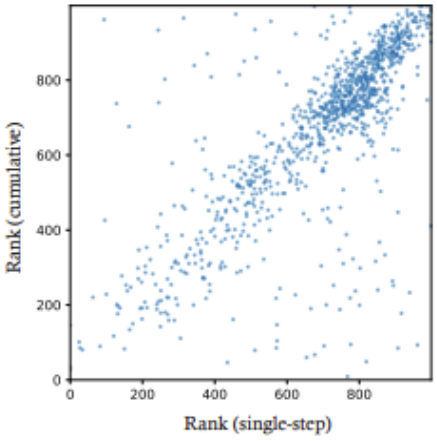
$$s_{l,c}^{(b)} = \mathbb{E}_{x \sim \mathcal{D}} \left[\left\| \tilde{\mathcal{A}}_{l,c}^{(b)}(\mathcal{V}, l) - \mathcal{A}^*(\mathcal{V}, l) \right\|_2^2 \right], \tag{4}$$

where $\tilde{\mathcal{A}}_{l,c}^{(b)}(x)$ is the perturbed action generated with the quantized channel, and $\mathcal{A}^*(x)$ is the reference action from the full-precision model. However, single-step error metrics like $s_{l,c}^{(b)}$ may not fully capture the error accumulation that occurs in long-horizon, autoregressive tasks. A small, initial error can compound over a sequence of actions. To account for this, we introduce a cumulative sensitivity metric, which measures the total deviation over an entire episode:

$$S_{l,c}^{(b)} = \mathbb{E} \left[\sum_{t=1}^{T} \left\| \tilde{\mathcal{A}}_{l,c}^{(b)}(\mathcal{V}_t, l) - \mathcal{A}^*(\mathcal{V}_t, l) \right\|_2 \right]. \tag{5}$$

This cumulative metric, $S_{l,c}^{(b)}$, naturally exhibits a stronger correlation with ultimate task success.

위와 같이 sensitivity를 기준으로 어떤 channel을 몇 bit로 양자화 할 지 결정



위 그림과 같이, single timestep과 full episode에 대한 sensitivity는 유사하기 때문에, full episode를 모두 누적하는 것보다 단순히 1 step에 대한 sensitivity를 계산

[QVLA]

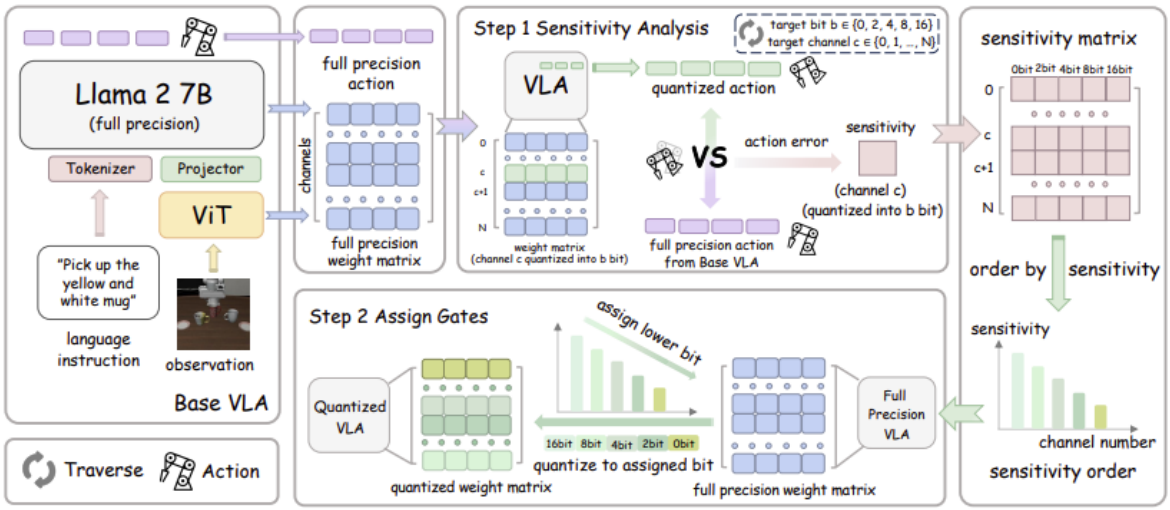


Figure 2: The pipeline of our **QVLA** framework consists of two steps: (i) In Step 1, we conduct a fine-grained action sensitivity analysis by systematically measuring and ranking the error induced by quantizing each channel to various bit-widths. (ii) In Step 2, an optimal bit-width is assigned to each channel using a greedy demotion algorithm, which iteratively prunes or lowers the precision of the least sensitive channels until the target bit budget is met.

QVLA의 과정은 매우 단순하다.

Step 1. 앞서 말한 channel 별 sensitivity를 계산한다. (1 time step sensitivity)

Step 2. 그리고 sensitivity가 낮은 순서부터 quantization을 진행한다. (16,8,4,2,0 한단계씩)

Step 3. Model Size Threshold까지 이를 반복한다.

PTQ를 진행하기 위해, calibration set은 다음과 같이 구축한다.

LIBERO와 같은 시뮬레이터 데이터나 real world에서 직접 수집한 trajectory data 등에 대해 각 타임스텝 별 (관측 이미지, 텍스트, action 라벨) 쌍을 가지는 여러 에피소드 데이터셋에서 랜덤 샘플링을 통해, 총 512쌍을 추출한다.

Sensitivity는 한 타임스텝 기준으로 정하기 때문에 서로 다른 에피소드가 들어와도 상관없다.

[Estimation of Action-Space Sensitivity]

한 타입스텝 기준이더라도, 여전히 모든 channel의 각각의 bit-width에 대한 sensitivity를 구하는 것은 cost가 매우 많이 든다.

A rigorous theoretical foundation is indeed crucial for justifying our design choices. We have incorporated detailed theoretical analysis into the revised manuscript, providing the derivation for our first-order proxy used to efficiently estimate the action-space sensitivity.

The true sensitivity we aim to minimize is the action space Mean Squared Error (MSE) when quantizing layer l, channel c to bit-width b:

$$S_{l,c}^{(b)} := \mathbb{E}[\|\Delta A\|^2], \quad \Delta A = A(X_{l,c} + \Delta X_{l,c}^{(b)}) - A(X_{l,c}),$$

위의 MSE 식이 true sensitivity이지만, 각 채널 별 sensitivity를 구하기 위해 최종 action 까지 모든 layer와 action head를 지나는 full forward pass에 대한 cost가 크다.

MSE 기반 민감 연산양: O(채널 개수 x 비트 후보 개수 x 전체 모델 추론 연산)

이에 처음에는 1차 Taylor 전개로 sensitivity를 coarse하게 구하고, 이후 이를 바탕으로 coarse sensitivity가 높은 channel에 대해서만 full forward pass로 fine grained 한 sensitivity를 구한다.

First-order Taylor expansion으로 MSE를 통한 sensitivity 계산을 다음과 같이 간소화할 수 있다.

where $A(\cdot)$ is the mapping from the channel output to the final action, and $\Delta X_{l,c}^{(b)}$ is the quantization noise.

Assuming a local linearity for small perturbations, we apply a first-order Taylor expansion to approximate the action deviation:

$$\Delta A \approx J_{A,X_{l,c}} \Delta X_{l,c}^{(b)},$$

where $J_{A,X_{l,c}}$ is the Jacobian of the action A with respect to the channel output $X_{l,c}$. Substituting this approximation into $S_{l,c}^{(b)}$ yields:

$$S_{l,c}^{(b)} \approx \mathbb{E}\big[\|J_{A,X_{l,c}} \Delta X_{l,c}^{(b)}\|^2\big] = \mathbb{E}\Big[(\Delta X_{l,c}^{(b)})^\top \underbrace{(J_{A,X_{l,c}}^\top J_{A,X_{l,c}})}_{=:H_{l,c}} \Delta X_{l,c}^{(b)}\Big].$$

We then employ the common uniform quantization noise model, characterized by zero mean and an isotropic covariance structure:

$$\mathbb{E}[\Delta X_{l,c}^{(b)}] = 0, \quad \mathrm{Cov}(\Delta X_{l,c}^{(b)}) = (\sigma_{l,c}^{(b)})^2 I.$$

Under this assumption, the expected quadratic form simplifies via the matrix trace property $\mathbb{E}[\mathbf{x}^\top H \mathbf{x}] = \mathrm{Tr}(H \mathrm{Cov}(\mathbf{x}))$:

$$\mathbb{E}\big[(\Delta X_{l,c}^{(b)})^\top H_{l,c} \Delta X_{l,c}^{(b)}\big] = \mathrm{Tr}\big(H_{l,c} \mathrm{Cov}(\Delta X_{l,c}^{(b)})\big) = (\sigma_{l,c}^{(b)})^2 \mathrm{Tr}(H_{l,c}).$$

Since $\mathrm{Tr}(H_{l,c}) = \mathrm{Tr}(J_{A,X_{l,c}}^\top J_{A,X_{l,c}}) = \|J_{A,X_{l,c}}\|_F^2$ (the squared Frobenius norm of the Jacobian), we arrive at the final first-order approximation:

$$S_{l,c}^{(b)} \approx (\sigma_{l,c}^{(b)})^2 \cdot \|J_{A,X_{l,c}}\|_F^2.$$

This theoretical derivation rigorously justifies our efficient proxy, which factors the sensitivity into a predictable quantization error term $(\sigma_{l,c}^{(b)})^2)$ and a task-dependent local sensitivity gain $(\|J_{A,X_{l,c}}\|_F^2)$.

1. Action 변화량에 대한 MSE를 quantization error와 이에 따른 Jacobian으로 변환
2. FIM에 의해 Jacobian의 제곱을 Hessian으로 동치
3. 단일 channel 내에서 quantization error는 평균=0, 분산이 균일한 등방성 구조
4. 이에 xHx를 Trace 연산으로 변환 후 등방성에 따라, sensitivity는 (분산 제곱) x (Jacobian 절대값의 제곱)으로 변환된다.
5. 즉, Full Forward Pass 없이, 모델의 Jacobian과 현재 target channel만 확인하면 sensitivity를 바로 구할 수 있다.

즉, 최종 sensitivity는 아래와 같이, target channel의 출력 변화와 이에 대한 Jacobian의 곱으로 정리할 수 있다.

$$\|\Delta \mathcal{A}\| \approx \|J_{\mathcal{A},\mathbf{X}_{l,c}}\| \cdot \|\Delta \mathbf{X}_{l,c}\|,$$

1차 Taylor 전개 기반 민감도 연산양: O(채널 개수 x 비트 후보 개수)
=> Jacobian은 연산 한 번으로 바로 구할 수 있음

[Optimal Bit Allocation]

이제, 각 채널 별 각 비트에 대한 sensitivity를 모두 구했다. 이를 바탕으로, 목표 memory budget에 맞게 bit allocation 진행.

모든 채널에 대해 모든 bit 조합에 대한 경우의 수를 고려하는 건, cost가 크다.

이에 greedy하게 비트 조합을 search

$b_{l,c} \in \{0, 2, 4, 8, 16\}$ to each channel to minimize the action error, subject to an average budget $\bar{B}$:

$$\min_{\{b_{l,c}\}} \sum_{l,c} s_{l,c}^{(b_{l,c})} \quad \text{s.t.} \quad \frac{1}{N} \sum_{l,c} b_{l,c} \leq \bar{B}, \tag{7}$$

where N is the total number of channels and 0-bit signifies pruning. To solve this NP-hard problem efficiently, we propose a greedy demotion algorithm. The procedure begins by initializing all channels to the highest precision, 16-bit. It then proceeds in a series of demotion stages ($16 \rightarrow 8$, $8 \rightarrow 4$, $4 \rightarrow 2$, and $2 \rightarrow 0$) until the average bit budget $\bar{B}$ is met. Within each stage, from a higher bit-width b_{hi} to a lower one b_{lo}, we evaluate the cost-effectiveness of demoting each candidate channel using the sensitivity-to-bit ratio $\rho_{l,c}$:

$$\rho_{l,c} = \frac{s_{l,c}^{(b_{\text{lo}})} - s_{l,c}^{(b_{\text{hi}})}}{b_{\text{hi}} - b_{\text{lo}}}. \tag{8}$$

<Greedy Search>

1. Full Precision 16 bit로 초기화
2. $\rho_{l,c}$: $lth\ layer,\ cth\ channel$의 현재 bit에서 한 단계 낮아질 때 sensitivity 변화율
   모든 layer에 대해 구한 후 해당 값을 min heap에 추가
   $\rho_{l,c}$가 크면 bit를 낮췄을 때 action sensitivity가 크다 즉, 오차가 커진다.
3. Min heap을 통해 가장 낮은 $\rho_{l,c}$ 부터 1 bit width만큼 강등. 즉, bit를 줄였을 때 **성능 변화가 적은 channel부터 quantization 적용**
4. 평균 bit width B에 도달할 때까지 반복

⇨ Min heap 자료구조에 따라 **O(ClogC)**의 bit allocation 소요

이를 바탕으로 sensitivity를 반영하여, projector와 action head는 full precision을 유지하면서, language model과 vision encoder의 둔감한 channel에 대해 quantization 진행

perform a limited number of full forward passes, selectively targeting the most important channels to precisely calibrate their true sensitivity scores. This hybrid approach allows us to focus computational effort on preserving sensitive interfaces, such as the projector and action head, while enabling aggressive compression of less critical channels.

Projector & Action head: Full-precision(BF16)

위의 channel 별 mixed bit allocation은 오직 weight quantization에만 적용

Activation은 uniform bit-width (8-bit)로만 quantization 진행

Weight는 모델 배포 전 이미 **고정된 값**. 각 가중치는 가 채널(행) 단위로 고유한 scale과 zero point를 가진 채 저장되며, 연산을 위해 메모리에 접근할 때 바로 역양자화 가능. 이미 고정된 값이기 때문에 채널마다 비트 수가 달라도 hardware-friendly

Activation은 추론이 진행되는 도중에 입력 데이터에 따라 **실시간으로 계속 계산되어 생성되는 값**. Activation의 bit-width가 layer마다 다르면, hardware는 연산을 진행하는 와중에 다음 층으로 넘어갈 때 매번 비트 포맷을 변환해서 해당 커널로 스위칭. Branching 없이 안정적인 연산을 위해 hardware-friendly하도록 uniform bit(8-bit)로 통일

[Experiment]

Table 1: **Performances under various weight-activation quantization settings**. W4A4/W8A8 refers to the quantization of weights (W) and activations (A) to 4 and 8 bits, respectively. Note that since our method assigns bits adaptively on a per-channel basis, we report the average bit-width.

Model	Setting	Method	Spatial	Object	Goal	Long	Avg ↑	Δ	Mem. (GB) ↓	Speedup ↑
OpenVLA	FP Model	-	84.7%	88.4%	79.2%	53.7%	76.5%	-	15.2	1×
	W8A8	SmoothQuant	84.2%	87.8%	77.8%	53.2%	75.8%	-0.7%	7.4	1.40×
		OmniQuant	82.6%	86.2%	74.8%	51.7%	73.8%	-2.7%	7.8	1.26×
		QVLA	85.2%	88.0%	77.6%	54.2%	76.3%	-0.2%	7.1	1.42×
	W4A4	SmoothQuant	69.2%	73.2%	69.6%	40.9%	63.2%	-13.3%	4.7	1.52×
		OmniQuant	82.2%	85.4%	75.4%	50.3%	73.3%	-3.2%	5.4	1.43×
		QVLA	84.4%	87.6%	78.8%	53.0%	76.0%	-0.5%	4.3	1.47×
OpenVLA -OFT	FP Model	-	97.6%	98.4%	97.9%	94.5%	97.1%	-	15.4	1×
	W8A8	SmoothQuant	96.4%	97.8%	95.4%	94.3%	96.0%	-1.1%	7.7	1.41×
		OmniQuant	95.4%	96.2%	93.0%	92.6%	94.3%	-2.8%	8.0	1.30×
		QVLA	97.2%	98.2%	95.8%	94.3%	96.4%	-0.7%	7.2	1.36×
	W4A4	SmoothQuant	77.2%	70.0%	77.8%	68.6%	73.4%	-23.7%	4.9	1.53×
		OmniQuant	95.0%	94.4%	94.0%	92.0%	93.9%	-3.2%	5.7	1.37×
		QVLA	96.2%	97.6%	96.4%	93.8%	96.0%	-1.1%	4.5	1.49×

Weight-Activation Quantization 결과

QVLA에서 가장 적은 성능 감소와 가장 작은 memory 사이즈 달성

속도는 SOTA는 아님.

Table 2: **Performances under various weight-only quantization settings**. Weight-only quantization primarily targets the reduction of memory, typically offering a marginal improvement in latency.

Model	Setting	Method	Spatial	Object	Goal	Long	Avg. ↑	Δ	Mem. (GB) ↓
OpenVLA	FP Model	–	84.7%	88.4%	79.2%	53.7%	76.5%	–	15.2
	W8A16	AWQ	82.3%	87.2%	74.6%	50.7%	73.7%	-1.8%	7.6
		QVLA	86.2%	88.4%	79.4%	53.1%	76.8%	+0.3%	7.2
	W4A16	AWQ	80.0%	81.2%	74.6%	47.2%	70.8%	-4.7%	5.0
		QVLA	86.0%	88.6%	78.4%	52.8%	76.5%	+0.0%	4.3
OpenVLA-OFT	FP Model	–	97.6%	98.4%	97.9%	94.5%	97.1%	–	15.4
	W8A16	AWQ	95.2%	96.8%	95.4%	93.1%	95.1%	-2.0%	8.0
		QVLA	97.4%	98.6%	97.2%	94.6%	97.0%	-0.1%	7.4
	W4A16	AWQ	93.0%	92.4%	93.8%	90.7%	92.5%	-4.5%	5.2
		QVLA	97.0%	98.4%	96.8%	94.4%	96.7%	-0.4%	4.5

Weight-only Quantization 결과

QVLA에서 가장 적은 성능 감소와 가장 작은 memory 사이즈 달성

Table 3: Comparison of layer-wise and channel-wise quantization methods on various tasks. The baseline is OpenVLA.

Weight Type	Method		Spatial	Object	Goal	Long	Avg. ↑
	Layer	Channel					
FP Model	–	–	84.7%	88.4%	79.2%	53.7%	76.5%
INT4	✓		84.2%	86.8%	77.0%	51.2%	74.8%
		✓	86.0%	88.6%	78.4%	52.8%	76.5%
INT8	✓		84.0%	87.2%	77.8%	50.6%	74.9%
		✓	86.2%	88.4%	79.4%	53.1%	76.8%

Layer가 아닌 Channel 별 quantization을 진행했을 때의 성능 비교

Action에 대한 sensitivity를 layer보다 더 작은 차원의 channel 수준으로 quantization을 진행해야 함을 검증

Table 4: **The influence of pruning (0-bit quantization) and uniform-bit quantization under an overall INT8 budget.** The row ④ with "prune" and without "uniform-bit" indicates our proposed quantization method with the $\{0, 2, 4, 8, 16\}$ as candidates during assigning bit to each channel.

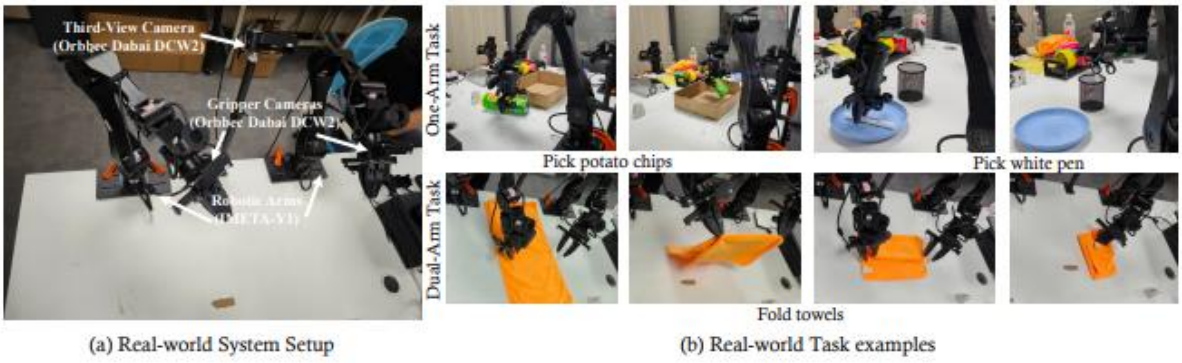
#	Weight Type	Quantization Method		Spatial	Object	Goal	Long	Avg. ↑	Memory (GB)
		Prune	Uniform Bit						
①	FP Model	–	–	84.7%	88.4%	79.2%	53.7%	76.5%	15.2
②	INT8			86.4%	88.0%	79.0%	53.4%	76.7%	7.5
③			✓	83.6%	87.0%	77.4%	50.2%	74.6%	7.6
④		✓		86.2%	88.4%	79.4%	53.1%	76.8%	7.0
⑤		✓	✓	84.0%	87.2%	77.0%	50.4%	74.7%	7.1

여기서 말하는 pruning은 특정 channel을 0-bit 즉, 0을 할당.

단순히 2bit까지 bit width를 줄이는 것뿐 아니라, $\rho_{l.c}$가 낮으면 0-bit로 pruning하는 것이 더 좋은 성능을 보임

따라서, 0-bit(prune) + channel-wise mixed bit 방식을 QVLA에 적용

<Real World 실험>



(a) Real-world System Setup (b) Real-world Task examples

Figure 4: Real-world system IMETA-Y1 and the task examples. The system consists of two IMETA-Y1 robotic arms and three Orbbec DaBai DCW2 cameras.

IMETA-Y1 로봇 팔 2대와 Depth camera 3대로 실험 진행. Task는 (1. 흰색 펜 줍기, 2. 과자 줍기, 3. 수건 접기) 3개로 구성되며, 해당 trajectory를 수집하여 학습. 해당 trajectory에서 random sampling하여 calibration set 구축하여 PTQ 진행.

Table 5: Real-world success rate comparison with AutoQVLA.

Method	Setting	One-Arm Task		Dual-Arm Task	Average	SpeedUp ↑
		Pick white pen	Pick potato chips	Fold towels		
π_0	-	8/10	7/10	4/10	63.3%	1.00×
AutoQVLA	W8A16	8/10	6/10	5/10	63.3%	1.28×

Results. We use the π_0 model as a real-world baseline and performed quantization under the W8A16 setting. Tab. 5 presents the results of the real-world experiments. We observe that AutoQVLA performs excellently on both single-arm and dual-arm tasks, with performance essentially on par with the original model. This strongly demonstrates the robustness of our quantization method, which maintains good performance even in real-world environments. Furthermore, we achieved an acceleration ratio of 1.28× under the W8A16 setting. Our model is trained on NVIDIA A100 GPUs and deployed for inference on a single NVIDIA 4070 GPU.

For additional technical depth, please refer to the Appendix, including the rigorous theoretical derivation of our sensitivity proxy and the complete algorithmic details. The Appendix also presents generalization results on the UniVLA architecture and CALVIN benchmark, alongside qualitative policy visualizations and ablation studies that further substantiate the robustness and architectural transferability of our framework.

[한줄평]

QVLA는 simple distribution-aware calibration 방식의 PTQ

Static한 weight에 대해 channel wise quantization을 적용

따라서, **방법론적으로나 모델 아키텍처적인 분석 기반의 quantization**에 대한 후속 연구의 여지 많아 보임.